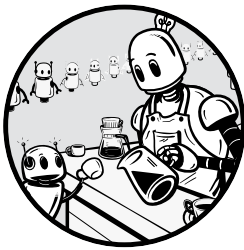


15

GRAPHS



Graphs are one of the fundamental data structures in computer science. They arise throughout numerous problems and programming tasks. Unlike the other data structures in this book, designed to optimize certain computations, the structure of *graphs* arises naturally from the data itself. In other words, graphs mirror the data they represent. Examining graph algorithms gives us insight into how we can define algorithms to utilize the inherent structure of the data.

Previous chapters focused on the problem of structuring the data to aid the algorithms; high-level problems, such as searching for a value, motivated and drove the design of the facilitating data structures. This chapter covers the opposite problem: graphs show us how the structure of the data can drive the development of new algorithms. In other words, given data in the form of a graph, we examine how to create algorithms that will use it. This chapter examines three graph algorithms that use different aspects of the graph's structure: Dijkstra's algorithm for shortest paths, Prim's algorithm for minimum-cost spanning trees, and Kahn's algorithm for topological sort.

Introducing Graphs

Graphs are composed of a set of *nodes* and a set of *edges*. As shown in Figure 15-1, each edge connects a pair of nodes. This structure is similar to a large number of real-world systems, including social networks (nodes are people and edges are their connections), transportation networks (nodes are cities and edges represent paths), and computer networks (nodes are computers and edges represent the connections between them). This variety of real-world analogs makes graph algorithms fun to visualize, as simple searches transform into careful exploration of castles or frantic sprints through a city's crowded alleys.

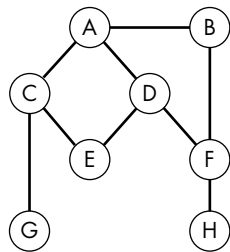


Figure 15-1: A graph with undirected edges

A graph's edges can have additional properties to capture the real-world complexities of the data such as whether or not the edges are directional. *Undirected edges*, like those in the graph in Figure 15-1, represent two-way relationships such as most roads and happy friendships. *Directed edges*, as illustrated in Figure 15-2, are like one-way streets and indicate a flow in a single direction. To represent undirected access, we use a pair of directed edges—one in each direction—between nodes. In a social context, directed edges could represent romantic interest in a television teen drama: an edge from Alice to Bob indicates Alice likes Bob, while the lack of an edge from Bob to Alice illustrates the devastating lack of reciprocity.

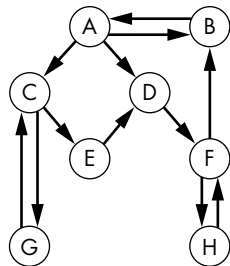


Figure 15-2: A graph with directed edges

In addition to allowing us to model one-way streets or unrequited love, directed edges allow us to model more abstract problems, such as task dependence. We can specify a set of tasks as nodes and use directed edges

to indicate the order dependency between tasks. In this way, we could create a graph to represent the tasks required for brewing the perfect cup of coffee, as shown in Figure 15-3. Nodes include such steps as heating the water, measuring out the beans, grinding the beans, and adding water to the grounds. The edges represent dependencies between these steps. We need to add a directed edge from the node for “grinding beans” to the node for “putting the grounds in the filter” to indicate that we must grind the beans first. The order of these two steps is critical, as anyone who has tried brewing unground beans can attest. However, we wouldn’t need an edge between “heating the water” and “grinding the beans” in either direction. We can perform those tasks in parallel.

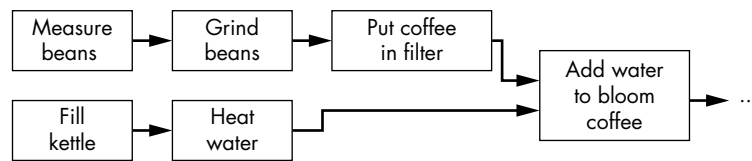


Figure 15-3: Using a graph to represent the order of operations for a task

Edge weights further increase the modeling power of graphs. *Weighted edges* capture not only the link between nodes but also the cost of that link. For example, we could weight the edges in a transportation graph by the distance between locations. We could augment our social network with a measure of closeness, such as a count of how many times two nodes have spoken in the last month. Figure 15-4 shows our example graph with weighted edges.

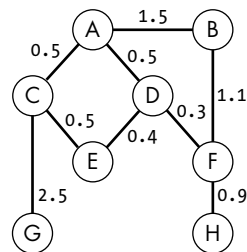


Figure 15-4: A graph with weighted edges

Using a combination of weighted and directed edges allows us to capture complex interrelations among the nodes. Entire social dramas can be represented and played out through the nodes and edges of a well-constructed graph.

Representing Graphs

While the abstract structure of a graph is relatively simple, there are multiple ways to represent nodes and edges in the computer’s memory. Two of the most common representations are *adjacency matrices* and *adjacency lists*. Both representations can handle directed, undirected, weighted, and

unweighted edges. As with all the other data structures in this book, the difference between these structures lies in how the data is stored in memory and thus how different algorithms can access it.

The adjacency list formulation stores a separate list of neighbors for each node. We could use an array or linked list of neighbors within the node composite data structure:

```
Node {  
    String: name  
    Array of Nodes: neighbors  
}
```

Or we could even create a separate edge data structure to store auxiliary information about the edges, such as their directionality or weights. For the examples below, we also provide a single numerical ID for each of the nodes, corresponding to the node's index in the parent graph data structure:

```
Edge {  
    Integer: to_node  
    Integer: from_node  
    Float: weight  
}  
  
Node {  
    String: name  
    Integer: id  
    Array of Edges: edges  
}
```

In either case, the graph itself would contain an array of nodes:

```
Graph {  
    Integer: num_nodes  
    Array of Nodes: nodes  
}
```

Regardless of the exact implementation, we can access the neighbors of any given node through a list linked from the node itself. Figure 15-5 shows an example of this structure.

In the case of directed edges, a node's list of edges or neighboring nodes contains only those that can be accessed when *leaving* the node. For example, node A may contain an edge to node B while B does not contain an edge to A.

Adjacency lists provide a localized view of neighbor relationships that mirrors real-world cases such as social networks. Each node tracks only the node to which it has connections. Similarly, in a social network, each person determines who qualifies as their friend, thus maintaining a list of their own connections. We don't need a separate central repository to tell us who our friends are, and we might not have a full view into anyone else's friends.

Arguably, we might not even know which of our friends (outgoing edge) actually consider us a friend in return (incoming edge). We know only about our own outgoing connections.

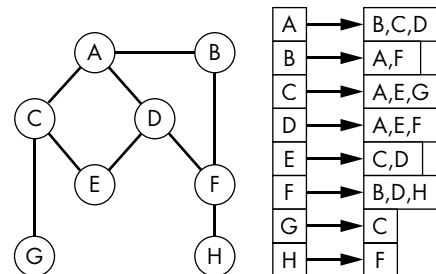


Figure 15-5: A graph (left) and its adjacency list representation (right). Each node stores a list of neighboring nodes.

In contrast, an adjacency matrix represents a graph as a matrix, as shown in Figure 15-6, with one row and one column for each node. The value in row i , column j represents the weight of the edge from node i to node j . A value of zero indicates that no such edge exists. This representation allows us to directly look up whether an edge exists between any two nodes from a single central data source.

	A	B	C	D	E	F	G	H
A	0	1	1	1	0	0	0	0
B	1	0	0	0	0	1	0	0
C	1	0	0	0	1	0	1	0
D	1	0	0	0	1	1	0	0
E	0	0	1	1	0	0	0	0
F	0	1	0	1	0	0	0	1
G	0	0	1	0	0	0	0	0
H	0	0	0	0	0	1	0	0

Figure 15-6: The adjacency matrix representation of a graph

This global view of the graph arises in real-world situations where a single planner is viewing the entire network. For instance, an airline company may use a global view of flight routes, where nodes are airports and edges indicate flights between them, to plan new service.

While the adjacency graph representation is useful in some cases, we will focus on the adjacency list representation for the remainder of this chapter. The list representation fits naturally with the pointer-based approach we've been using for other data structures. Further, the use of individual node data structures allows additional flexibility in terms of storing auxiliary data.

Searching Graphs

If we look back to our web-crawling example from Chapter 4, where we explored our favorite online encyclopedia for information related to coffee

grinders, we can immediately see how the links in our favorite online encyclopedia form a graph of topics, with each page representing a node and each hyperlink representing a directed edge. We can progressively explore topics, diving deeper and deeper into the world of coffee grinders, by iteratively exploring each node and adding new nodes onto our list of topics to explore in the future. This type of exploration forms the basis of a graph search.

Imagine that we are interested in finding a specific node in the graph. Perhaps we are conducting online research and looking for a coffee brand whose name we have long forgotten. We explore the relevant web pages (graph nodes) one at a time, reading the information on one page before moving to another. As we saw in Chapter 4, the order in which we explore the nodes greatly influences our search pattern. By using a stack data structure to track our future exploration options, we conduct a depth-first search over the graph. We pursue individual paths deeper and deeper until we hit a dead end. Then we backtrack and try other options we skipped along the way. If we instead use a queue to track our future search states, we perform a breadth-first search over the nodes. We check the nodes closer to our starting location before venturing further and further into the graph. Of course, there are a variety of other ways we could order our search. For example, best-first search orders the future nodes according to a ranking function, focusing on exploring high-scoring nodes first. In our search for nearby coffee shops in a new city, this prioritization of nodes can keep us from wasting hours wandering through residential neighborhoods instead of focusing on commercial areas.

Regardless of the order, the concept of searching a graph by exploring one node at a time illustrates the impact of the data's structure on the algorithm. We use the links between nodes (edges) to constrain and guide the exploration. In the next few sections, we look at common useful algorithms that do exactly this.

Finding Shortest Paths with Dijkstra's Algorithm

Probably the single most common task when dealing with real-world graphs is to find the shortest distance between two nodes. Imagine we're visiting a new city for the first time. As morning dawns, we stumble out of our hotel room, jetlagged and in search of refreshment. As good travelers, we've done copious research on the city's coffee scene and created a list of four coffee shops to sample while in town. As the elevator reaches the lobby, we pull out a street map of the city, carefully marked with the location of the hotel and those coffee shops. It's time to determine how to get to the shops on our list.

Dijkstra's algorithm, invented by the computer scientist Edsger W. Dijkstra, finds the shortest path from any given starting node to all other nodes in the graph. It can work on directed, undirected, weighted, or unweighted graphs. The only constraint is that all the edge weights must be non-negative. You can never decrease the total path length by adding an edge. In our coffee-themed sightseeing example, we search for the shortest path from the hotel to each of the coffee shops. As shown in Figure 15-7, nodes represent either street intersections or shops along the street. Weighted, undirected edges represent the distance along the roads between these points.

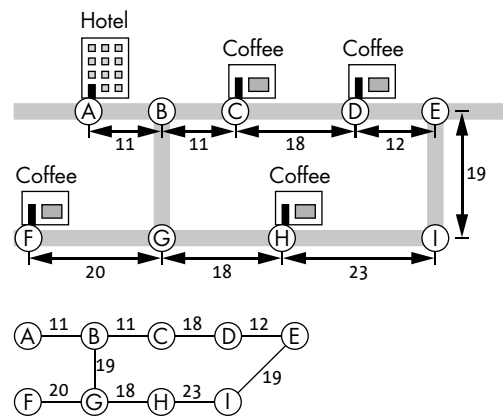


Figure 15-7: Points along a map with corresponding distances (top) can be represented as a weighted graph (bottom).

Our goal is to find the shortest path from the starting node to each of the coffee shop nodes. The intersection nodes aren't goals in their own right but allow our path to branch over different streets.

Dijkstra's algorithm operates by maintaining a set of unvisited nodes and continually updating the *tentative* distance to each unvisited node. At each iteration, we visit the closest unvisited node. Upon doing so, we remove this new node from our unvisited set and update the distances to each of its unvisited neighbors. Specifically, we examine the new node's neighbors and ask whether we have found a better path to each neighbor. We compute the length of the new proposed path by taking the distance to the current node and adding the distance (edge weight) to the neighbor. If this new distance is less than the best distance seen so far, we update the distance.

```

Dijkstras(Graph: G, Integer: from_node_index):
  ❶ Array: distance = inf for each node id in G
    Array: last = -1 for each node in G
    Set: unvisited = set of all node indices in G
    distance[from_node_index] = 0.0

  ❷ WHILE unvisited is NOT empty:
    ❸ Integer: next_index = the node index in unvisited
                          with the minimal distance
    Node: current = G.nodes[next_index]
    Remove next_index from unvisited

    ❹ FOR EACH edge IN current.edges:
      ❺ Float: new_dist = distance[edge.from_node] +
                        edge.weight
      ❻ IF new_dist < distance[edge.to_node]:
        distance[edge.to_node] = new_dist
        last[edge.to_node] = edge.from_node
  
```

The code starts by creating a series of helper data structures ❶, including an array of distances to each node (*distance*), an array indicating the last node visited before a given node (*last*), and a set of unvisited nodes (*unvisited*). The code then processes the unvisited nodes one by one. A **WHILE** loop iterates until the set of unvisited nodes is empty ❷. In each iteration, the code chooses the node with the minimal distance and removes it from the unvisited set ❸. A **FOR** loop iterates over each of the node's neighbors ❹, computing the distance to that neighbor through the current node ❺ and updating the distance and last arrays if the code has found a better path ❻.

NOTE

While both the pseudocode and illustrations use an explicit distance array and unvisited set for the sake of illustration, these can be combined into min-heap for efficiency. The min-heap functions as a priority queue that always returns the lowest-priority item. In this case, it stores the list of unvisited nodes, keyed by their current distance. Finding the closest node consists of returning the minimum key in the queue, and updating the best distance so far corresponds to updating a node's priority.

Figure 15-8 shows an example shortest-path search from node A in Figure 15-4's weighted graph. The circled node is the one currently being examined. The grayed-out nodes and list entries represent nodes that have been removed from the unvisited list and thus are no longer available for consideration.

For the search in Figure 15-8, we start Dijkstra's algorithm with all distances at infinity except for node A, which is set to zero (Figure 15-8(1)). This starting configuration corresponds to our initial knowledge about the best paths. We are already at node A, so the best path there is trivial. Since we have not found paths to any of the other nodes, they could be any distance away. We also maintain information for each node of which node precedes it in our search. The *last* column indicates the preceding node. This information allows us to trace paths backward. While not all uses will need to reconstruct the path, our coffee search certainly does. It is pointless to find the shortest distance to coffee if we don't also find the actual path. To construct the path to node F, we follow the last pointers back until we reach node A.

Our search starts, as shown in Figure 15-8(2), by selecting the node with the smallest distance (node A), removing it from the unvisited list, and examining its neighbors. For each of A's neighbors, we test whether traveling to that neighbor through A is shorter than any path seen so far. Since the distance to Node A is zero, the distance through A to each of its neighbors will be equal to the corresponding edge weights. Each time we update the distance to an unvisited node, we also update the back pointer to reflect the best path so far. Three nodes now point to A (Figure 15-8(2)).

The search progresses, choosing the next closest, unvisited node. In this case, it could be either C or D. We break the tie using the node's order in our list: node C wins! Again, we consider C's neighbors and update their best distances (Figure 15-8(3)). Remember the distances represent the best total distance from our starting node. The new distances are the sum of the distance to C and the distance from C to each neighbor.

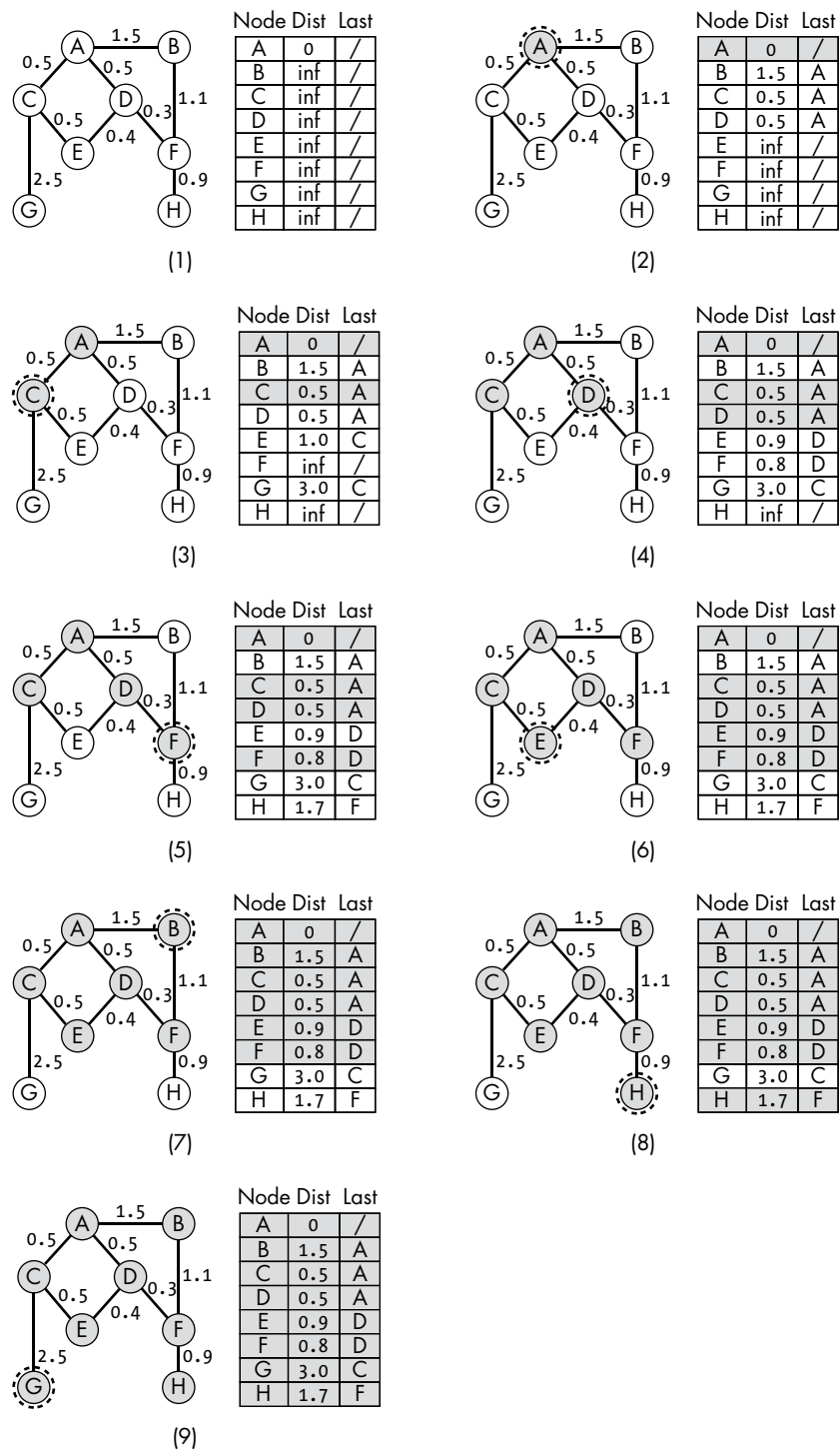


Figure 15-8: An example of Dijkstra's algorithm on a weighted graph

The search progresses to node D—the new unvisited node with the minimum distance (Figure 15-8(4)). While examining node D's neighbors, we find new shortest distances to both nodes E and F. Node E is particularly interesting, as we already had a candidate path to E through C. We can travel from A to C to E with a distance of 1.0. However, this is not the best possible path. Our search revealed a new path, through D, that is slightly shorter with a total distance of 0.9. We update both the potential distance and the backward pointer. Our best path to E now goes through D. On to the next closest node in our unvisited set, node F!

The search continues through the remaining nodes, but nothing else interesting occurs. The remaining nodes are all at the end of the shortest paths and don't offer opportunities for shorter paths. For example, when considering node E's neighbors (Figure 15-8(6)), we examine both nodes C and D. The distance to either node when traveling through E would be 1.4, longer than the paths we've already discovered. In fact, both C and D have already been visited, so we wouldn't even consider them. Similar logic applies when considering nodes B, H, and G as shown in Figure 15-8(7), 15-8(8), and 15-8(9). Since those nodes' neighbors have all been visited, we do not consider them.

In examining how Dijkstra's algorithm traverses a graph while finding the shortest path, we can see the clear interrelation between the structure of the data and the algorithm itself. Shortest-path algorithms like Dijkstra's are only necessary because of the structure of the problem. If we could effortlessly hop from any node to any other node, there would be no need to find a path along the edges. This is the real-world equivalent of teleporting from our hotel lobby to the target coffeeshop—convenient, but not allowed by the structure of the physical world. Thus, while searching for these shortest paths, we need to obey the structure of the graph itself.

Finding Minimum Spanning Trees with Prim's Algorithm

The problem of finding the *minimum spanning tree* of a graph provides another example of how the structure of graph data enables us to ask new questions and thus create new algorithms suited to answering these questions. The minimum spanning tree of an undirected graph is the smallest set of edges such that all of the nodes are connected (if possible). We can think of these trees in terms of a budget-conscious city planner, trying to determine which roads to pave. What is the absolute minimal set of roads needed in order to ensure that anyone can get from one place (node) to any other place (node) on a paved road? If the edges are weighted, such as by the distance or the cost of paving a road, we extend the concept to finding the set that minimizes the total weight: the *minimum-cost spanning tree* is the set of edges with the minimum total weight that connect all the nodes.

One method of finding the minimum spanning tree is *Prim's algorithm*, which was independently proposed by multiple people, including computer scientist R. C. Prim and mathematician Vojtěch Jarník. The algorithm operates very similarly to Dijkstra's algorithm in the previous section, working through an unvisited set and building up a minimum spanning tree one

node at a time. We start with an unvisited set of all nodes and randomly choose one to visit. This visited node forms the start of our minimum spanning tree. Then, on each iteration, we find the unvisited node with the minimum edge weight when compared to *any* of the nodes that we've previously visited. We are asking, "Which node is closest to our set's periphery and thus can be added with the least cost?" We remove this new node from the unvisited set and add the corresponding edge to our minimum-cost spanning tree. We keep adding nodes and edges, one per iteration, until every node has been visited.

We can picture Prim's algorithm as a construction company hired to build bridges between islands in an archipelago. The builders start at a single island and work outward, connecting more and more islands. At each step, they choose the closest island to the ones in the currently connected set. One end of the bridge sits on an island in the connected set and one end sits on an island outside the connected set (bringing the new island into the connected set). By always starting new bridges from an island in the connected set, the builders are able to move their equipment to the starting island using the existing bridges. And by always ending bridges on islands outside the connected set, the builders increase the coverage of the connected set at every stage.

We can simplify the algorithm's code by tracking additional information. At each step, we maintain a list of the best edge (including weight) that we have encountered to each node. Every time we remove a new node from the unvisited set, we examine that node's unvisited neighbors and check whether there are better (i.e., lower-cost) edges to any of its neighbors. If there are, we update the neighbor's entry in the list with the new edge and weight.

Prims(Graph G):

```

❶ Array: distance = inf for each node in G
   Array: last = -1 for each node in G
   Set: unvisited = set of all node indices in G
   Set: mst_edges = empty set

❷ WHILE unvisited is NOT empty:
    ❸ Integer: next_id = the node index in unvisited with
                        the minimal distance
    ❹ IF last[next_id] != -1:
        Add the edge between last[next_id] and
        next_id to mst_edges
        Remove next_id from unvisited

        Node: current = G.nodes[next_id]
    ❺ FOR EACH edge IN current.edges:
        IF edge.to_node is in unvisited:
            IF edge.weight < distance[edge.to_node]:
                distance[edge.to_node] = edge.weight
                last[edge.to_node] = current.id
return mst_edges

```

The code starts by creating a series of helper data structures ❶, including an array of distances to each node (*distance*), an array indicating the last node visited before a given node (*last*), a set of unvisited nodes (*unvisited*), and the final set of edges for the minimal spanning tree (*mst_edges*). As with Dijkstra's algorithm, the pseudocode (and the figures we'll discuss in a moment) use a combination of lists and sets for the sake of illustration. We can more efficiently implement the algorithm by storing the unvisited nodes in a min-heap keyed by the distance. For now, we will list all the values in order to explicitly illustrate what is happening.

The code then proceeds like Dijkstra's algorithm, processing the unvisited nodes one at a time. A **WHILE** loop iterates until the set of unvisited nodes is empty ❷. During each iteration, the node with the minimal distance to any of the visited nodes is chosen and removed from the unvisited set ❸. The code checks whether an incoming edge to the node exists, which is necessary because the first node visited will not have an incoming edge ❹, and adds the corresponding edges to the minimum spanning tree. After adding the new node, a **FOR** loop iterates over each of the node's neighbors ❺, checking whether the neighbor is unvisited and, if so, checking its distance to the current node. In this case, the distance is simply the weight of the edge. The code finishes by returning the set of edges making up the minimum spanning tree.

Consider what happens when we run Prim's algorithm on the weighted graph from Figure 15-4, as illustrated in Figure 15-9. We start with all *last* edges set to null (we have not found any yet) and all "best" distances to infinity. For simplicity's sake, we'll break ties in alphabetical order of the nodes.

To begin, we remove the first node A from our unvisited set. We then consider all of A's neighbors and check whether there is a lower-cost edge from A to that neighbor. Given that all our current best distances are infinity, this isn't difficult. We find lower-cost edges for all of A's neighbors: (A, B), (A, C), and (A, D). Figure 15-9(1) shows this new state.

During the second iteration, we find two potential nodes in our unvisited set to use: C and D. Using alphabetical order to break the tie, we select C. We remove C from the unvisited set and add the edge (A, C) to our minimum-cost spanning tree. Examining C's unvisited neighbors, we find better candidate edges to nodes E and G (Figure 15-9(2)).

The next closest node is D. We remove that from our unvisited set and add the edge (A, D) to the minimum-cost spanning tree. When we examine D's unvisited neighbors, we find new, lower-cost edges to both nodes E and F (Figure 15-9(3)). Our best candidate edge to node E now originates from node D instead of node C.

The algorithm progresses through the remaining nodes in our unvisited set. Next, we visit node F, adding the edge (D, F), as shown in Figure 15-9(4). Then, as shown in Figure 15-9(5), we add node E and edge (D, E). The algorithm completes by adding nodes H, B, and G in that order. At each step, we add the corresponding best edge seen so far: (F, H), (F, B), and (C, G). The final three steps are shown in Figure 15-9(6), Figure 15-9(7), and Figure 15-9(8), respectively.

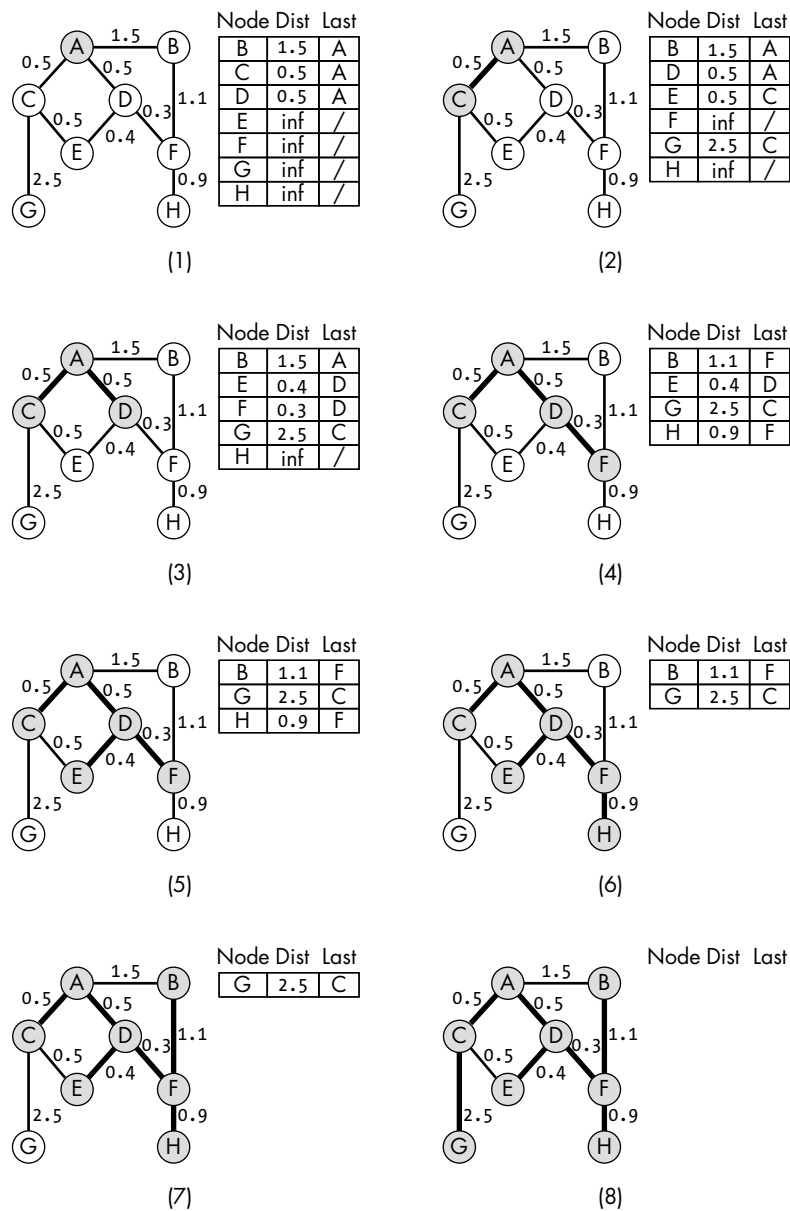


Figure 15-9: An example of Prim's algorithm on a weighted graph

Prim's algorithm doesn't care about the total path lengths from our starting node. We're only interested in the cost of adding the new node to our connected set—the edge weight that will link that node to any other node in the visited set. We are not optimizing for final drive times between nodes, just for minimizing the cost of paving roads or building new bridges.

What if we had broken ties randomly instead of by alphabetical order? When deciding between choosing nodes D or E from our unvisited set after Figure 15-9(2), we could have used either one. If we had chosen E instead of

D, we would have found a lower-cost edge weight linking D into our graph. The algorithm would link in node D through E rather than through A. This means that we can find different minimum-cost spanning trees for the same graph. Multiple different trees may have the same cost. Prim's algorithm only guarantees that we find one of the trees with the minimal cost.

Topological Sort with Kahn's Algorithm

Our final example of a graph algorithm uses the edges of a *directed acyclic graph* (DAG) to sort the nodes. A directed acyclic graph is a graph with directed edges arranged such that the graph contains no *cycles*, or paths that return to the same node, as shown in Figure 15-10. Cycles are critical in real-world road networks. It would be terrible if roads were constructed such that we could get from our apartment to our favorite coffee shop but could never navigate back. Yet this is exactly what happens in an acyclic graph—the path out of any node will never return to that same node.

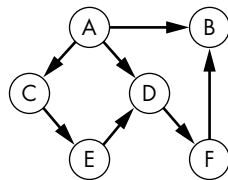


Figure 15-10: A directed acyclic graph

We can use directed edges to indicate an ordering of the nodes. If the graph has an edge from A to B, node A must come before node B. We ordered nodes in this way in our coffee-brewing example at the beginning of the chapter: each node represented a step in the process, and each edge indicated one step's dependency on the next. The person brewing the coffee has to perform a given step before they can perform any of the following steps. These types of dependencies arise throughout both computer science and the rest of life. An algorithm that sorts the nodes in order of their edges is called a *topological sort*.

Computer scientist Arthur B. Kahn developed one approach, now called *Kahn's algorithm*, to perform topological sort on a directed acyclic graph representing events. This algorithm operates by finding the nodes with no incoming edges, removing them from our list of pending nodes, adding them to our sorted list, and then removing the outbound edges from that node. The algorithm repeats until we have added every node to our sorted list. Intuitively, this sort mirrors how we might perform a complex task in the real world. We start with a subtask that we can accomplish—one with no dependencies. We perform that subtask and then choose another to do. Any subtask that requires us to have performed a yet uncompleted task needs to wait on our list until we have finished all its dependencies.

When implementing Kahn's algorithm, we don't need to actually remove edges from our graph. It's sufficient to keep an auxiliary array counting the number of incoming edges to each node and modifying those counts.

```

Kahns(Graph G):
    ❶ Array: sorted = empty array to store result
    Array: count = 0 for each node in G
    Stack: next = empty stack for the next nodes to add

    # Count the incoming edges.
    ❷ FOR EACH node IN G.nodes:
        FOR EACH edge IN node.edges:
            count[edge.to_node] = count[edge.to_node] + 1

    # Find the initial nodes without incoming edges.
    ❸ FOR EACH node IN G.nodes:
        IF count[node.id] == 0:
            next.Push(node)

    # Iteratively process the remaining nodes without
    # incoming connections.
    ❹ WHILE NOT next.IsEmpty():
        Node: current = next.Pop()
        Append current to the end of sorted
        ❺ FOR EACH edge IN current.edges:
            count[edge.to_node] = count[edge.to_node] - 1
            ❻ IF count[edge.to_node] == 0:
                next.Push(G.nodes[edge.to_node])

    return sorted

```

The code starts by creating several helper data structures ❶, including an array to hold the sorted list of nodes (sorted), an array storing the count of incoming edges for each node (count), and a stack of the next node to add to sorted (next). The code uses a pair of nested FOR loops over the nodes (outer loop) and each node's edges (inner loop) to count the number of incoming edges for each node ❷. Then a FOR loop over the count array finds nodes that have no incoming edges and inserts them into next ❸.

The code then uses a WHILE loop to process the next stack until it is empty ❹. During each iteration, the code pops a node off the stack and adds it to the end of the sorted array. A FOR loop iterates over the node's edges and reduces the count (effectively removing the incoming edge) for each neighbor ❺. Any neighbor with an incoming count of zero is added to next ❻. Finally, the code returns the array of sorted nodes.

If our graph does contain cycles, our sorted list will be incomplete. We may want to add an additional check at the end of the function to test that the number of elements in our sorted list equals the number of nodes in the graph.

Consider running this algorithm on the graph from Figure 15-10, as is illustrated in Figure 15-11. We start off by counting the number of incoming edges (shown as the number adjacent to each node) and determining

that node A is the only node without any incoming edges (Figure 15-11(1)). Kahn's algorithm then adds A to the sorted list and removes its outgoing edges (by decreasing the corresponding counts), as shown in Figure 15-11(2).

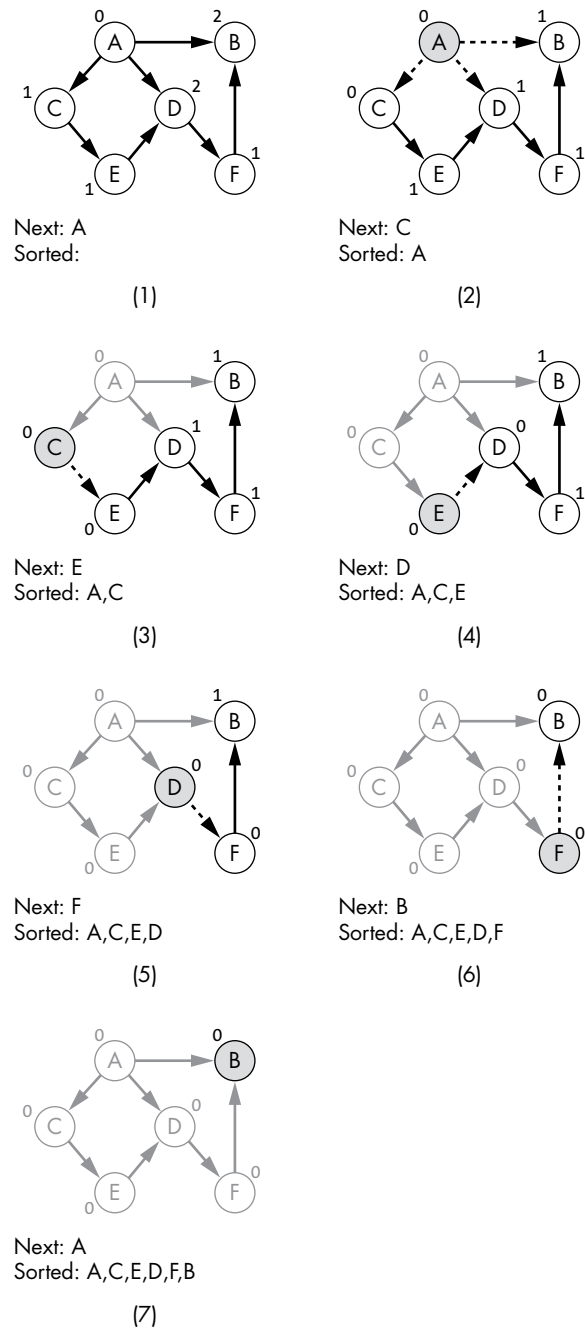


Figure 15-11: A topological sort on a directed acyclic graph

We continue the algorithm on node C (Figure 15-11(3)), which no longer has any incoming edges. We removed the only such edge when we processed node A. We remove C from our list of nodes under consideration (our stack next), remove its edges from the graph, and add it to the end of our sorted list. In the process, we've left node E without any incoming neighbors. E goes onto our stack.

The sort progresses through the remainder of the list. While processing node E, we remove the last incoming edges to node D, making it the next up for the algorithm (Figure 15-11(4)). The sort then adds D, then F, then B to our sorted list as shown in Figure 15-11(5), Figure 15-11(6), and Figure 15-11(7), respectively.

Kahn's algorithm presents an example of both the usefulness of directed edges in a graph and how we can design an algorithm to operate on them. The directionality of the edges further constrains how we explore nodes.

Why This Matters

Graphs are pervasive throughout computer science. Their structure allows them to mirror a large variety of real-world phenomena, from streets to social or computer networks to sets of complex tasks. Graphs are useful for tasks like path planning and determining the order in which to compile a program's source code. There are a myriad of algorithms designed to operate over these data structures, performing such tasks as searching the graph, determining the minimum spanning tree, or determining the maximum flow through a graph. We could devote an entire book to this single vastly impactful data structure.

For the purposes of this chapter, however, we focus on the tight coupling between the structure of the data and the algorithms that operate on it. The graph structure of data drives new problems, such as finding the minimum spanning tree, and thus new algorithms. In turn, the algorithms use the graph structure of the data, traversing the edges and exploring from node to node. This interplay demonstrates the importance of understanding the structure of data when defining both problems and new solutions.